

BUILD PROMPT — paste this into Claude Code (or any coding agent)

Use this after the project docs (PRD.md, ARCHITECTURE.md, TECH_STACK.md, TEAM_ROLES.md, memory.md) are in the repo root. Run it from the repo root so the agent can read them.

Full prompt

You are building "SIM Swap & Transaction Fraud Risk Analytics" – a hybrid rules+ML fraud scoring prototype for banks. Before writing any code, read these files in the repo root in full: PRD.md, ARCHITECTURE.md, TECH_STACK.md, TEAM_ROLES.md, memory.md. Treat them as your spec – do not invent features they don't describe, and do not deviate from the locked stack in TECH_STACK.md (Python/Flask backend, SQLite, scikit-learn logistic regression, JWT auth, React or plain HTML/CSS/JS frontend – pick one and state which).

Build in this order, and after each step run whatever tests exist and show me the result before moving to the next step:

1. Scaffold the exact repo layout from

ARCHITECTURE.md §3 (backend/api,
backend/risk_engine,
 backend/db, frontend, simulators,
notifications, tests). Create requirements.txt
and

.env.example matching TECH_STACK.md §3. Do
not add folders that aren't in that layout.

2. backend/db/: write schema.sql for the tables
in ARCHITECTURE.md §4 (users, customers,
 sim_events, transactions, otps, alerts,
agents), a models.py with thin data-access
 functions (no business logic), and seed.py
that inserts a handful of demo customers.

3. backend/risk_engine/: implement rules.py
(pure function: event dict in, {score, reasons:
 []}) out – rules for SIM-change frequency,
time-since-SIM-change, new device, location
 mismatch, failed-login count, transaction
amount), features.py (turns an event into a
 feature vector), ml_model.py + train_model.py
(logistic regression trained on generated
 synthetic labeled data – write a small
synthetic data generator if none exists yet),
and
 config.py (ALLOW/STEP_UP/BLOCK thresholds).
risk_engine must have zero Flask or SQL
 imports – it's called by the API layer, never
the other way around.

4. backend/api/: Flask app with JWT auth
(register/login), POST /api/risk/evaluate (calls
 risk_engine, persists via db/models.py,
returns decision + reasons), POST
/api/otp/verify,
 GET /api/dashboard/summary, POST

/api/agents/heartbeat. Validate every request body and
return clear 4xx errors on bad input. Update the API contract table in memory.md as you finalize each route's request/response shape.

5. notifications/: a notifier that sends SMS (Twilio) and email (SMTP/SendGrid) alerts when a BLOCK decision or OTP is created, reading credentials from env vars – if the vars are unset, log to console instead of raising, so the system still runs without real keys.

6. simulators/: a traffic generator that posts realistic login/transaction/SIM-change events to the running API at an adjustable rate, and an agent heartbeat script that pings POST /api/agents/heartbeat every N seconds.

7. frontend/: a dashboard that polls GET /api/dashboard/summary and shows recent transactions, SIM events, alerts, risk scores and reasons, and agent online/stale status, plus a simple screen to submit an OTP for a STEP_UP transaction.

8. tests/: pytest coverage for risk_engine (rule and threshold behavior) and for the API endpoints (happy path + a couple of validation/error cases). Add a short end-to-end test or script that: registers a user, submits a high-risk transaction, confirms it's BLOCKed and an alert row exists.

Constraints:

- Never hardcode secrets; only read them from

```
environment variables listed in
  TECH_STACK.md §3.
- Keep risk_engine, api, db, frontend,
simulators, and notifications in their own
folders per
  ARCHITECTURE.md §3 – don't mix concerns across
folders.
- After finishing each numbered step, stop and
summarize what you built and any open
  question before continuing, rather than doing
all 8 steps silently in one pass.
- At the end, update memory.md's changelog with
what was built and any decisions you made
  that weren't already pinned in the docs (e.g.
exact rule weights you chose).

Start with step 1.
```

If you're running this per-teammate instead of all at once

Each of the four members can run a scoped version of the same prompt, replacing the "Build in this order" section with just their owned step(s) from TEAM_ROLES.md, and adding: *"Only create or edit files inside your owned folder(s); if you need a function from another module, write it as a documented stub with a clear TODO and note the required signature in memory.md's API contract table."* That keeps agents (or people) working in parallel without stepping on each other's files.

Tip

Run this in a repo that already has Git initialized and the docs committed, so the agent can diff its own changes and you can review each step's commit before merging.